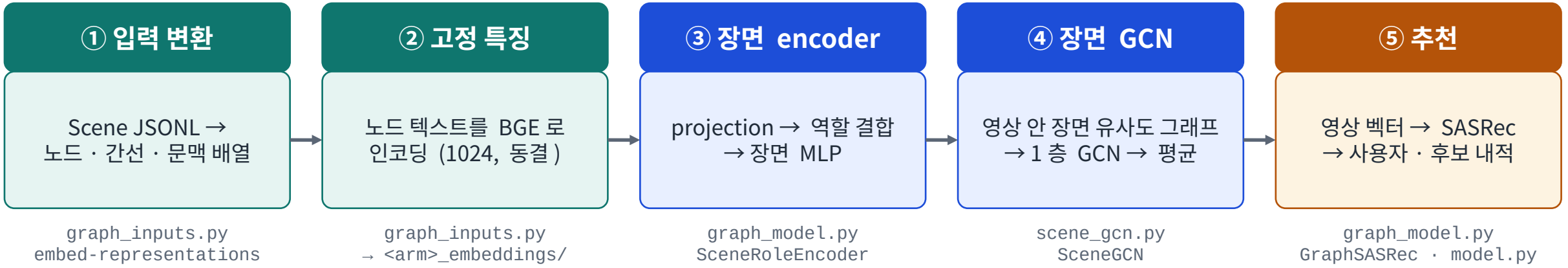
Scene GCN 은 어떻게 구현되어 있나

ViewingContextPipeline · --representation-mode graph --scene-aggregation gnn
모델 계약 sasrec-role-graph/v8 · 코드 기준 2026-10-06

src/validation/graph_inputs.py · graph_batching.py · graph_model.py · scene_gcn.py · graph_context.py · model.py

전체 흐름 : Scene Graph 파일에서 추천 점수까지 5 단계



학습 시 ③④⑤는 하나의 모델 (GraphSASRec) 이며 , 추천 loss 의 gradient 가 ⑤ → ④ → ③까지 흐릅니다 . ①② 는 임베딩 단계에서 한 번만 계산해 저장합니다 .

학습되는 부분 (추천 loss 로 갱신)

- 4 개 projection (Title · Entity · Action · Context, 1024→128)
  - 역할 결합 Linear (640→128) 과 none/unknown 상태 벡터
- Scene Residual MLP (512), GCN Linear (512→512)
- SASRec Transformer, 위치 임베딩 , User MLP

고정된 부분

- BGE large-en-v1.5 (텍스트 → 1024). 학습 중 갱신하지 않음
  - 그래프 구조 : 어떤 Entity 가 어떤 Action 의 어느 역할인지
  - 장면 묶음 인덱스 (scene_groups): 영상별 장면 수로 결정
  - 장면 유사도 Â 는 고정이 아님 — 매 forward 다시 계산

1. 입력 : Scene Graph 가 텐서가 되기까지

graph_inputs.py · embed-representations 단계 · 출력 validation/representations/graph/<arm>_embeddings/

장면 1 개의 Scene Graph → 노드 · 간선 · 문맥 (graph_inputs.py)

```
[Context]
medium: live_action
format: demonstration
topics: food preparation
[Entities]
p1: person; wearing apron
f1: tomato
t1: knife
l1: kitchen
[Actions]
p1 - cutting - f1; t1; l1
```

v8 Scene Graph 출력 (파싱되어 scene_graph 객체로 저장)

배열	이 장면에서 생기는 값	뜻
nodes / types	`p1, f1, t1, l1 → type 0` `cutting → type 1`	노드 = Entity(0) 또는 Action(1). 값은 BGE 에 넣을 텍스트의 ID

edges	`(p1→cutting, actor=0)`
	`(f1→cutting, target=1)`
	`(t1→cutting, tool=2)`
	`(l1→cutting, location=4)`
	`+ 역방향 4 개 (role+5)`
		Entity → Action 간선 .
		역방향은 저장만 하고 모델은 쓰지 않음

missing	`[cutting, −1, −1, −1, 0, −1]`	역할별 상태 . −1 = 연결됨 , 0 = none, 1 = unknown. receiver(4 번째) 는 항상 0

contexts	`"Format: demonstration`
	`Medium: live_action`
	`Topics: food preparation"`
		장면당 1 개의 문맥 텍스트

BGE 에 들어가는 텍스트 (ID 는 넣지 않음)

```
Entity
"Name: person
Attributes: wearing apron"
```

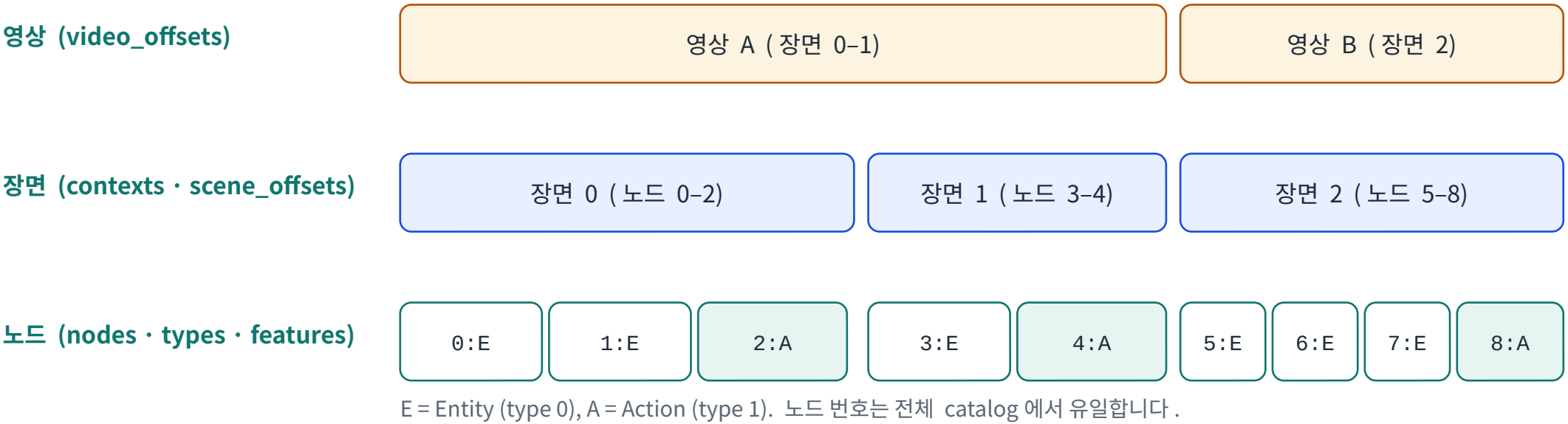
```
Action
"cutting"
```

```
Context
"Format: …\nMedium: …\nTopics: …"
```

같은 텍스트는 한 번만 인코딩합니다 (text_id 로 중복 제거). 제목은 Arm 이 제목을 쓰는 경우에만 영상당 1 개 text_id 로 들어가고 , 아니면 0(없음) 입니다 .
구조 검사 (validate_graph_structure) 에 실패하거나 warning 이 있는 장면은 제외되고 excluded_scenes 에 기록됩니다 .

저장 형식 : 영상 → 장면 → 노드의 들쭉날쭉한 배열을 offsets 로 이어 붙임

예 : 영상 A 는 장면 2 개 (노드 3+2), 영상 B 는 장면 1 개 (노드 4). 모든 영상의 장면 · 노드가 한 줄로 이어지고 , 경계는 offsets 배열에만 있습니다 .



E = Entity (type 0), A = Action (type 1). 노드 번호는 전체 catalog 에서 유일합니다 .

```
video_offsets  = [0, 2, 3]        # 영상 i 의 장면 : [v[i], v[i+1])
scene_offsets  = [0, 3, 5, 9]     # 장면 s 의 노드 : [o[s], o[s+1])
edge_offsets, missing_offsets     # 같은 방식
titles         = [text_id_A, 0]   # 영상별 제목 text_id (0 = 없음 )
features       = ( 고유 텍스트 수 × 1024) float32, 동결 BGE 출력
```

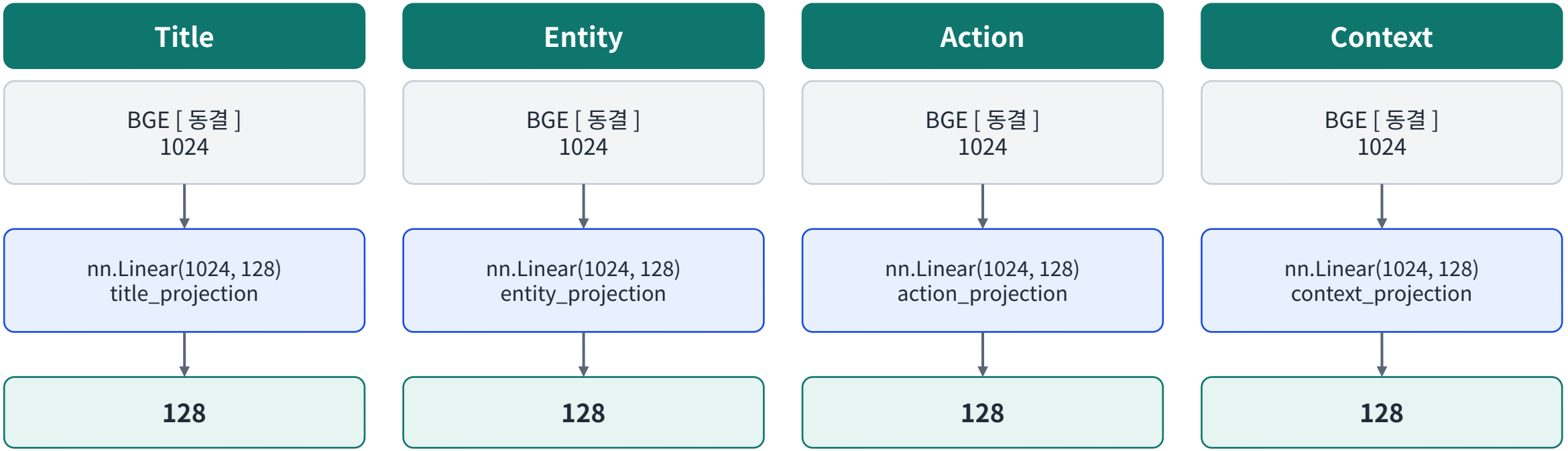
PackedGraphStore (graph_batching.py)

- 학습 시작 때 video→node, video→edge 경계와 역할별 간선 목록을 미리 계산
- 배치의 영상 ID 를 받아 그 영상들의 장면 · 노드 · 간선을 한 번의 벡터 연산으로 모음 (ranges)
- BGE 특징 전체 (features) 를 GPU 에 올릴 수 있으면 상주시킴 (feature_cache_mb, 기본 2048MB)

2. 모델 : SceneRoleEncoder.forward 를 한 단계씩

graph_model.py 62–118 행 · scene_gcn.py · 입력은 배치 하나 (여러 영상의 장면 · 노드), 출력은 영상당 512 차원 벡터

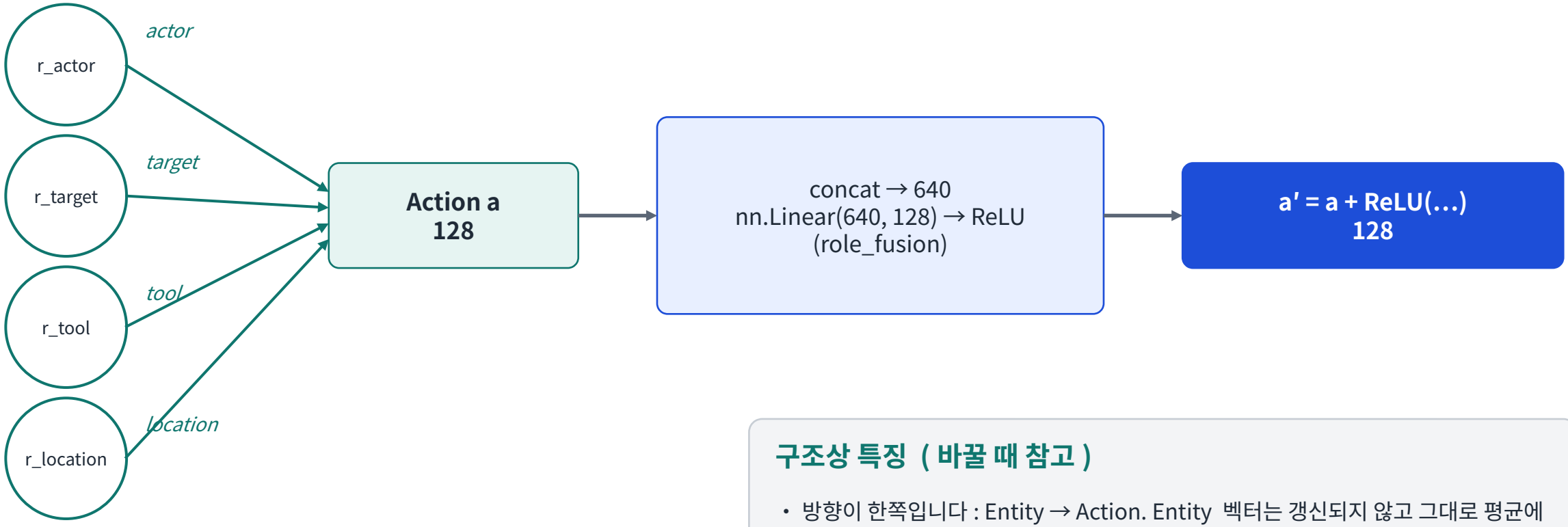
단계 1 · Projection: 네 종류 입력을 각각 1024 → 128 로



코드에서 확인할 점

- 네 projection 은 가중치를 공유하지 않습니다 (종류별 별도 Linear). bias 포함 각 131,200 개 파라미터 .
- 제목은 title_available 마스크로 0 처리합니다 . 즉 ' 제목 없음 ' 은 Linear 를 거친 뒤 0 벡터가 됩니다 (graph_model.py 65–66 행).

단계 2 · 역할 결합 : Entity 정보를 Action 벡터에 한 번 더함



r_ 역할 = 연결된 Entity 의 128 차원 벡터 (단계 1 출력)
연결이 없으면 → 학습 가능한 상태 벡터

```
role_states: nn.Parameter(4, 2, 128)
[ 역할 4] × [none=0, unknown=1] × 128
```

구조상 특징 (바꿀 때 참고)

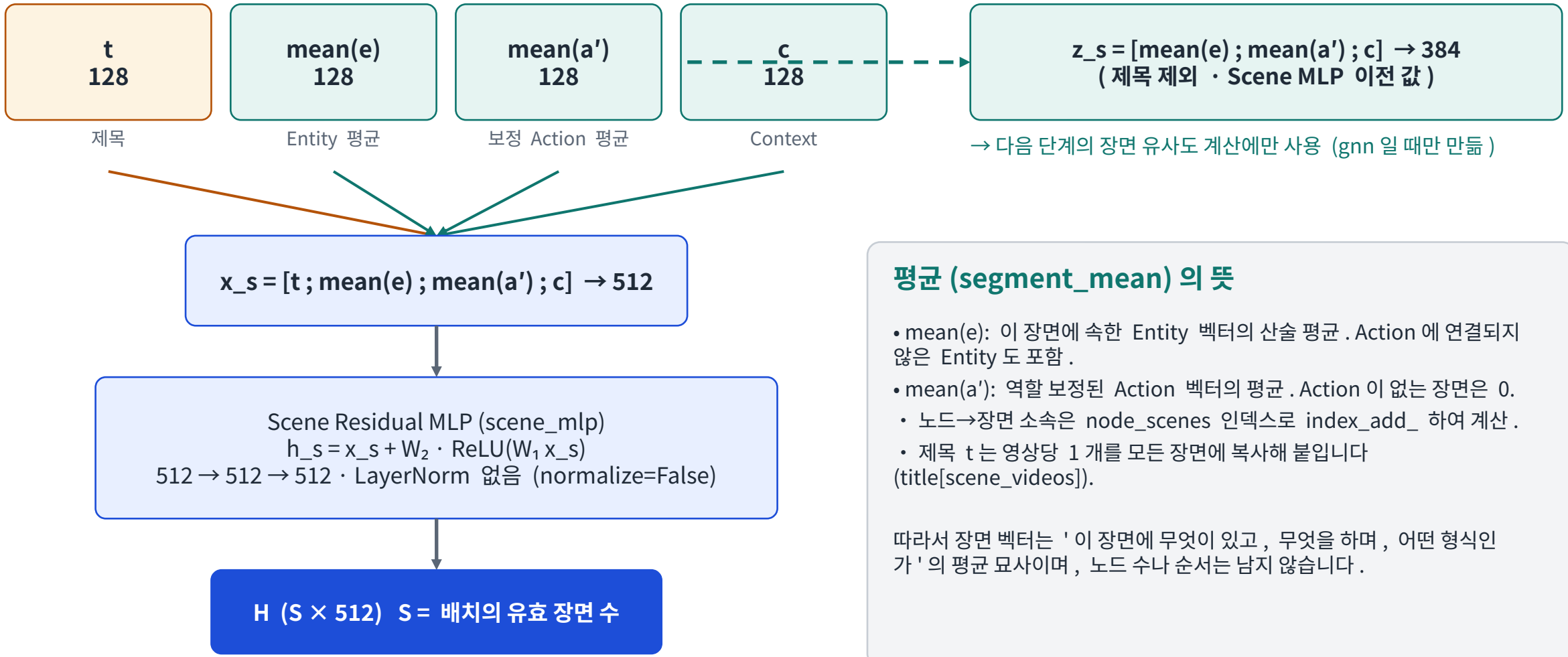
• 방향이 한쪽입니다 : Entity → Action. Entity 벡터는 갱신되지 않고 그대로 평균에 들어갑니다 .

• 1-hop 1 회입니다 . 층을 쌓거나 반복하지 않습니다 .

• receiver 역할은 입력에서 항상 ' 없음 ' 으로 들어오고 모델도 읽지 않습니다 (ROLE_INDICES = 0,1,2,4).

• 역할마다 Entity 가 최대 1 개입니다 (v8 형식 : actor - action - target; tool; location).

단계 3 · 장면 벡터 : 네 성분 concat(512) → Residual MLP

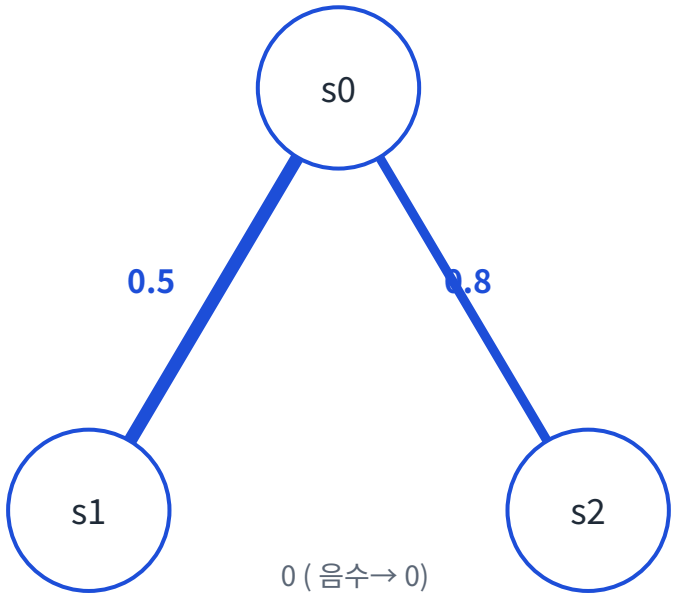


평균 (segment_mean) 의 뜻

- mean(e): 이 장면에 속한 Entity 벡터의 산술 평균 . Action 에 연결되지 않은 Entity 도 포함 .
- mean(a′): 역할 보정된 Action 벡터의 평균 . Action 이 없는 장면은 0.
- 노드→장면 소속은 node_scenes 인덱스로 index_add_ 하여 계산 .
- 제목 t 는 영상당 1 개를 모든 장면에 복사해 붙입니다 (title[scene_videos]).

따라서 장면 벡터는 ' 이 장면에 무엇이 있고 , 무엇을 하며 , 어떤 형식인가 ' 의 평균 묘사이며 , 노드 수나 순서는 남지 않습니다 .

단계 4a · 영상 안 장면 유사도 그래프 Â



cos(z₁,z₂) 가 0 이하면 간선이 사라집니다 .
self-loop(A_ii = 1) 는 그림에서 생략 .
모든 장면 쌍을 보므로 장면 S 개면 S^2 항 .

A = max(cos, 0), 대각 1

1	.80	.50
.80	1	0
.50	0	1

D = 행합 (대각 행렬)

2.30			.435	.393	.269
	1.80		.393	.556	0
		1.50	.269	0	.667

Â = D^-½ A D^-½

Â_ij = A_ij / √(D_i · D_j)

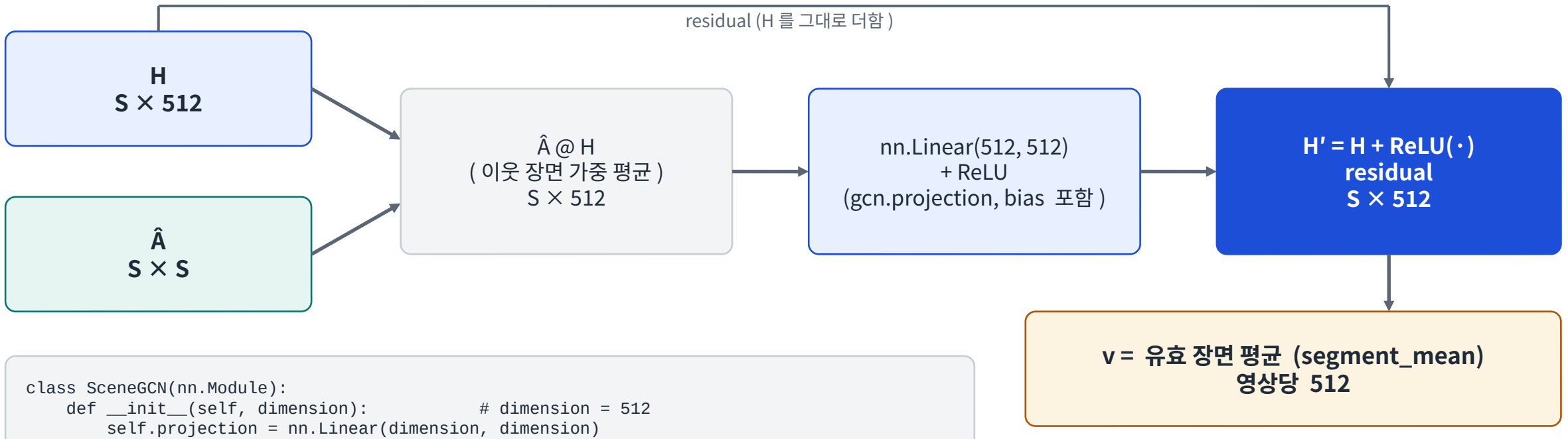
예 : Â_01 = 0.80 / √(2.30 × 1.80) = 0.393

```
unit    = F.normalize(z, dim=-1, eps=1e-8)          # 영벡터 보호
weights = (unit @ unit.T).clamp_min(0)              # 음수 cosine → 0
weights = where(eye, 1, weights)                    # self-loop = 1
scale   = weights.sum(-1).rsqrt()                   # D^-1/2
Â       = weights * scale[:, None] * scale[None, :]
```

핵심 성질

- Â 는 학습 파라미터가 아니라 현재 z 에서 매 forward 계산됩니다 . z 는 projection· 역할 결합을 거치므로 gradient 가 Â 를 통해서도 흐릅니다 (detach 없음).
- temperature·top-k·threshold 가 없어 cosine 이 모두 높으면 Â 의 모든 항이 비슷해집니다 .

단계 4b · GCN 1 층과 영상 벡터 : H′ = H + ReLU(ÂHW + b)




```
class SceneGCN(nn.Module):
    def __init__(self, dimension):           # dimension = 512
        self.projection = nn.Linear(dimension, dimension)
    def forward(self, scenes, similarity_features, groups):
        output = torch.zeros_like(scenes)
        for indices in groups:                    # 장면 수가 같은 영상 묶음
            adjacency = normalized_similarity(similarity_features[indices])
            values = scenes[indices]              # ( 영상 수 , S, 512)
            updated = values + relu(self.projection(adjacency @ values))
            output = output.index_copy(0, indices.reshape(-1), updated.flatten(0, 1))
        return output
```

유사도는 Scene MLP 이전의 z(384) 로 계산하지만 , 전파되는 값 H 는 Scene MLP 이후의 512 차원입니다 . 즉 ' 누가 이웃인가 ' 와 ' 무엇을 전달하는가 ' 의 입력이 다릅니다 .

이것이 GCN 의 전부입니다

- 층 1 개 , 가중치 행렬 1 개 (512×512 + bias = 262,656 개)
- 활성 ReLU, residual 있음 , LayerNorm·dropout·gate 없음
- 전파 후 평균이므로 장면 순서 정보 없음
- 장면 1 개 영상 : Â=[1] → h + ReLU(hW + b)

예외 경로 : 장면이 없거나 제목이 없는 영상은 어떻게 처리되나

유효 장면 ⩾ 1	장면 0 + 제목 있음	장면 0 + 제목 없음	Arm = meta
일반 경로	fallback (GCN 건너뜀)	0 벡터	GCN 없음
역할 결합 → Scene MLP → GCN → 평균 . 장면 1 개도 self-loop GCN 을 거칩니다 .	[t ; 0 ; 0 ; 0] (512) 을 같은 scene_mlp 에 통과 . GCN 을 거치지 않으므로 출력 분포가 다를 수 있음 . (graph_model.py 112–117 행)	available 마스크로 최종 출력 0. 추천 점수는 항상 0 → 사실상 추천 불가 . 결측 때문에 catalog 에서 빼지는 않음 .	GraphSASRec 이 아니라 일반 SASRec(제목 1024 → 512 → MLP). 같은 graph 모드로 실행해도 장면을 전혀 읽지 않음 .

어떤 장면이 ' 유효 ' 한가 (graph_inputs.py 134–155 행)

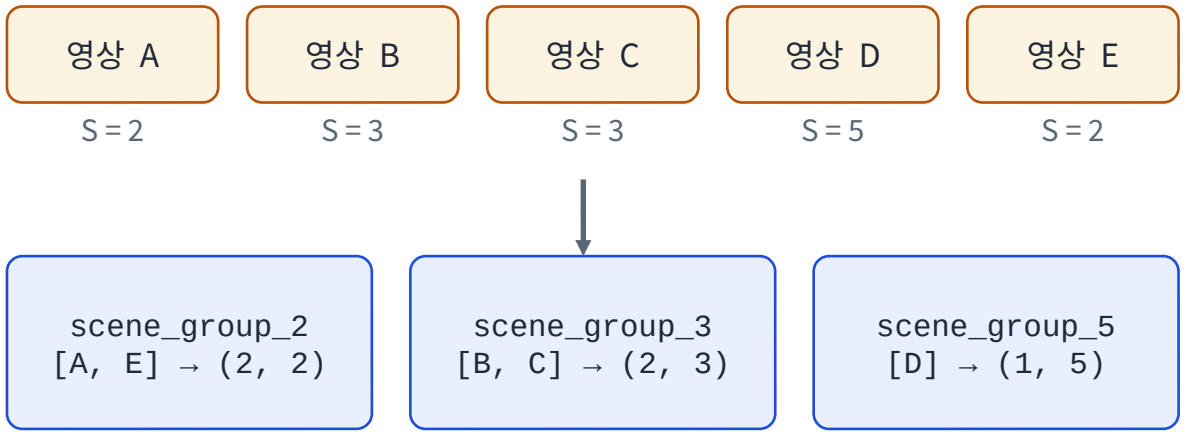
- scene_graph 가 dict 가 아니면 (파싱 실패 원문) → raw 로 제외
- warning 필드가 있으면 → warning 으로 제외 (내용과 무관하게 제외)
- actions 키가 없으면 unsupported_format, validate_graph_structure 실패면 invalid_structure 로 제외
- 통과한 장면은 Action 이 0 개여도 유효 장면입니다 (actionless 통계만 기록). 제외 사유는 <arm>_embeddings/ 의 statistics 에 남습니다 .

3. 실행과 학습 : 배치는 어떻게 만들고 무엇으로 학습하나

graph_batching.py · graph_model.py GraphSASRec · model.py in_batch_loss · rolling_recommendation.py

배치 처리 : 장면 수별 묶음과 S^2 기준 chunk 분할

scene_groups(offsets) — 배치 안 영상을 장면 수별로 묶음



각 묶음은 (영상 수 , S) 장면 인덱스 행렬입니다 . 묶음마다 Â 를 (영상 수 , S, S) 로 한 번에 계산하고 배치 행렬곱으로 전파합니다 . 영상 사이에는 간선이 생기지 않습니다 (묶음 안에서도 영상별로 독립된 Â).
인덱스는 CPU 에서 만들어 배치에 넣습니다 (group_scenes=True).

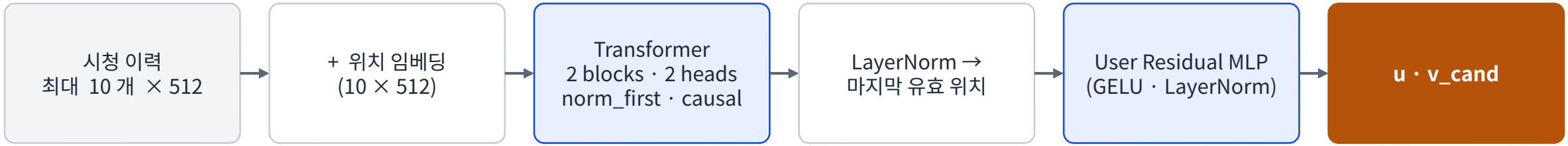
chunk 기준 (config validation.graph_execution)	기본값	역할
chunk_items	256	chunk 당 최대 영상 수
chunk_nodes	32,768	chunk 당 최대 노드 수
chunk_scene_pairs (gnn 만)	1,048,576	chunk 당 Σ S^2 상한 . 한 영상이 넘으면 단독 처리
checkpoint	auto	활성 메모리 추정이 크면 gradient checkpoint 사용
feature_cache_mb	2,048	BGE 특징을 GPU 에 상주시킬 상한

학습 1 step 에서 일어나는 일 (GraphSASRec)

1. 배치의 시청 이력 · 정답에 나오는 영상 ID 를 모아 unique 처리
2. chunk 로 나눠 SceneRoleEncoder 실행 → 영상 벡터 캐시 (_vectors)
3. SASRec 이 이력 위치마다 캐시에서 벡터를 찾아 사용자 벡터 계산
4. loss → backward: gradient 가 encoder 까지 전달
5. optimizer step 후 캐시 폐기 (train() 호출 시 clear_item_cache)

평가 때는 catalog 전체 (19,738 개) 를 한 번 인코딩해 catalog_vectors 로 둡니다 .

추천 모델과 손실 : 영상 벡터 → SASRec 사용자 벡터 → 내적



후보 영상 벡터 v_cand 는 같은 SceneRoleEncoder 출력이며 Transformer 를 거치지 않습니다 . 사용자와 영상이 같은 512 차원 공간에서 내적으로 비교됩니다 .

학습 손실 in_batch_loss (model.py 253 행 ~)

- 이력의 모든 위치에서 다음 영상을 맞히는 next-item 예측
- 후보 = 같은 배치의 정답 영상들 (in-batch negatives)
- logits = u · v_cand − log p(인기도) ← logQ 보정 (popularity_power 1.0)
- 사용자가 이미 본 영상과 자기 정답 외 중복은 − 1e4 로 마스킹
- cross-entropy (softmax over batch candidates)

최적화 : AdamW lr 1e-4 · weight decay 0.1 · batch 128 · gradient clipping · deterministic

단계	데이터 구간	역할
Selection	D−1 일 이전 학습 , [D−1, D) 검증	검증 NDCG@10 으로 epoch 수 선택 (patience 5, max 50)
Refit	D 이전 전체	모델 ·optimizer 초기화 후 선택한 epoch 만큼 다시 학습
Test	[D, D+1)	이벤트별 순위 ·HR·NDCG@K (K = 4, 8, 10, 20, 30)

평가일 7 일 × seed 3 개 (42, 43, 44) × Arm 마다 독립 학습 . 각 조합은 recommendations/graph/gnn/<date>/seed_<seed>/<arm>/ 에 sasrec.pt · training.json · per_event_metrics.jsonl · catalog_vectors.npy 로 저장됩니다 .

mean 과 gnn 의 차이는 GCN 한 층뿐입니다

항목	--scene-aggregation mean	--scene-aggregation gnn
모델 계약 (graph_context.py)	sasrec-role-graph/v7 · GRAPH_MODEL	sasrec-role-graph/v8 · GNN_MODEL (gcn 설정 포함)
장면 encoder	projection → 역할 결합 → Scene MLP	동일
장면 → 영상	유효 장면 평균	유사도 $\hat{A}$ 계산 → 1 층 residual GCN → 유효 장면 평균
추가 파라미터	없음	gcn.projection 512×512 + 512 = 262,656 개
추가 입력	없음	z_s (384) · scene_group_* 인덱스 (CPU 에서 생성)
chunk 기준	items · nodes	items · nodes · ΣS^2 (chunk_scene_pairs)
결과 경로	recommendations/graph/mean/ · graph_mean_diagnosis.json	recommendations/graph/gnn/ · graph_gnn_diagnosis.json
고정 Graph 입력	representations/graph/<arm>_embeddings/ 공유 (role-graph-inputs/v3)	동일 — 재임베딩 불필요

비용 참고 (docs/graph_training.md, 합성 입력 · 영상 64 개 · 장면 315 개 · L4 GPU): mean 21.1 ms/batch, gnn 44.6 ms/batch (약 2.1 배). 실제 데이터의 장면 수 분포 (평균 약 6 개) 에 따라 달라지며 , 추천 품질 차이는 아직 측정되지 않았습니다 .

학습 파라미터 수 (코드로 모델을 만들어 센 값)

1,395,840
SceneRoleEncoder (gnn)

262,656
그중 GCN 층

6,837,248
GraphSASRec 추천부

8,233,088
gnn 모델 합계

모듈	모양	파라미터	비고
title/entity/action/context_projection	4 × Linear(1024, 128)	524,800	종류별 별도 가중치
role_fusion	Linear(640, 128)	82,048	Entity→Action 결합
role_states	Parameter(4, 2, 128)	1,024	none/unknown 상태
scene_mlp	2 × Linear(512, 512)	525,312	residual, LayerNorm 없음
gcn.projection	Linear(512, 512)	262,656	gnn 에서만 존재
position_embedding	Embedding(10, 512)	5,120	시청 순서
encoder (Transformer)	2 × EncoderLayer(512, ff 2048)	6,304,768	SASRec 본체
norm + user_mlp	LayerNorm + 2 × Linear(512, 512)	527,360	사용자 벡터

mean 모델 합계는 7,970,432 개입니다 . BGE(약 3.35 억) 는 동결이며 포함하지 않았습니다 . 추천부의 item_projection·item_mlp 는 GraphSASRec 에서 삭제됩니다 .

4. GNN 구조를 바꾸려면

고칠 파일 · 캐시가 재사용되지 않게 하는 방법 · 현재 구조에서 보이는 특성과 미검증 가설

바꿀 파일과 반드시 함께 바꿀 것

파일	무엇이 있나	바꿀 때
validation/scene_gcn.py	normalized_similarity, SceneGCN	인접 행렬 정의 (temperature·top-k· 시간 간선), 층 수 , attention 등 GCN 본체
validation/graph_model.py 84–111	z_s 구성 , GCN 호출 위치 , 평균 풀링	유사도 입력 바꾸기 , GCN 을 Scene MLP 앞으로 , attention 풀링 등
validation/graph_context.py 23–43	GNN_ARCHITECTURE, GNN_MODEL	필수 : 새 버전 문자열과 gcn 설정 기록 . 캐시 키 ·checkpoint metadata 에 들어감
validation/recommendation_contracts.py 6	DIAGNOSIS_ARCHITECTURE_VERSIONS	필수 : 새 버전 추가 . 없으면 run-diagnosis 가 결과를 거부
validation/cli.py 37 · graph_context.py	--scene-aggregation choices, validate_mode	새 이름 (예 : gnn_v2) 을 추가할 때
tests/validation/test_scene_gcn.py 등	GCN 수치 ·gradient· 격리 검사	기존 수치 기대값은 바꿈 . 새 구조에 맞춰 갱신

왜 버전 문자열을 꼭 바꿔야 하나

추천 결과는 identity(encoder·item_model·graph_model·graph_architecture 등) 의 해시로 shared_cache 에 저장됩니다 . 구조만 바꾸고 GNN_MODEL 을 그대로 두면 해시가 같아 이전 GNN 결과가 학습 없이 복원될 수 있습니다 (recommendation_cache.py 28–35 행).

권장 : 새 aggregation 이름으로 추가

결과 폴더가 recommendations/graph/<aggregation>/ 이므로 , 기존 gnn 을 직접 고치면 같은 Run 안에서 이전 결과를 덮어씁니다 . gnn_v2 같은 이름을 추가하면 mean · gnn · gnn_v2 를 한 Run 에서 나란히 비교할 수 있고 고정 Graph 입력은 공유합니다 .

현재 구조에서 보이는 특성 (코드로 확인) 과 개선 가설 (미검증)

유사도가 고르면 GCN 은 MLP 한 층과 같음

Â 의 모든 항이 1/S 에 가까우면 Â·H ≈ 평균이 되어 v ≈ mean(H) + ReLU(W·mean(H) + b). 그래프 구조가 결과에 기여하지 않음 . 같은 영상의 장면은 Context 가 비슷해 이럴 가능성이 있으나 실제 Â 분포는 측정하지 않음 .

시간 정보 없음

장면 순서를 잇는 간선도 , 위치 인코딩도 없음 . 장면 순서를 섞어도 영상 벡터가 같음 .

음수 유사도는 0 으로 잘림

대비되는 장면 사이 정보는 쓰지 않음 . 날카롭게 만드는 temperature·top-k 도 없음 .

역할 그래프는 한 방향 1-hop

Entity 는 Action 에서 정보를 받지 않음 . 역방향 간선은 저장만 됨 .

개선 가설 — 효과는 아직 실험하지 않았습니다

- Â 에 temperature softmax 또는 top-k 적용
- 학습 가능한 attention 간선 (GAT 방식)
- 시간상 인접 장면 간선 추가 또는 위치 인코딩
- GCN 2 층 이상 · 평균 대신 attention 풀링
- Entity 도 갱신하는 양방향 역할 메시지 전달
- 구조 변경 전에 실제 Â 의 비대각 가중치 · 행 entropy 측정

요약

- GNN = 장면 안 ' 역할 결합 1-hop' + 영상 안 ' 유사도 GCN 1 층 ' + 평균 풀링
- Â 는 제목을 뺀 384 차원 cosine, 매 forward 재계산 , 파라미터 없음 · 학습 파라미터는 GCN Linear 512×512 하나
- 구조를 바꾸면 graph_context.py 의 버전 · 설정과 DIAGNOSIS_ARCHITECTURE_VERSIONS 를 함께 바꿔야 캐시 재사용을 피함